학습 노트: DB 릴레이 (에이전트 간 결과 전달)

작성일: 2026-06-06 **관련 파일**: `dev2/db.py`, `dev2/nodes/aggregate.py`, `dev2/nodes/concept_mapper.py`, `dev2/schema.py` **복습 예정**: D+1 (6/7), D+3 (6/9), D+7 (6/13)

> 이 노트는 한 세션에서 작업한 3가지를 묶었습니다. **메인 주제는 ③ DB 릴레이**이고, ①②는 그 과정에서 함께 고친/만든 것입니다.

---

③ DB 릴레이 — 메인 주제

한 줄 요약

에이전트 A(Dev2)가 분석 결과를 **공용 창고(DB)** 에 넣어두면, 에이전트 B(Dev4)가 **송장번호(task_id)** 로 그 결과를 찾아가서 쓰는 방식입니다.

4-Layer 설명

1. 비유 🚚 vs 🏪

- **기존 방식 (payload)**: 택배기사(M0)가 Dev2의 물건을 **직접 손에 들고** Dev4에게 가져다줍니다. 기사가 중간에 사라지거나 물건을 안 들고 오면 Dev4는 아무것도 못 받습니다.
- **DB 릴레이 방식**: Dev2가 물건을 **공용 창고(DB)** 에 넣고 송장번호(task_id)를 붙입니다. Dev4는 송장번호만 알면 **언제든 창고에 가서 직접 꺼내** 옵니다. 기사가 필요 없습니다.

2. 개념

"DB 릴레이"는 에이전트 간 데이터 전달을 **메모리/API payload가 아니라 데이터베이스를 경유**해서 하는 패턴입니다. 보내는 쪽은 DB에 쓰고(write), 받는 쪽은 **공통 키(여기선 `task_id`)** 로 DB에서 읽습니다(read). 두 에이전트는 서로를 직접 호출하지 않고 DB만 공유합니다.

3. 왜 쓰나요?

기존에는 Dev2가 결과를 **두 군데로 따로** 보냈습니다.

- DB에 저장 (기록·통계용)
- payload로 M0에게 전달 → M0가 Dev4에게 전달 (실제 사용)

→ **같은 데이터가 두 경로로 흐르는데 서로 연결이 안 돼 있었습니다.** Dev2가 DB에 저장한 결과를 Dev4가 쓰지 않았던 거죠. DB 릴레이로 바꾸면 "저장한 그 데이터를 그대로 받아 쓰니까" 경로가 하나로 합쳐지고, M0가 중간에서 결과를 들고 다닐 필요가 없어집니다.

4. 실제 코드

```
# dev2/db.py — 보내는 쪽이 저장한 걸 받는 쪽이 task_id로 조회
async def fetch_scan_results_by_task(task_id: str) -> list[dict]:
    # scan_executions ⋈ scan_issues 를 task_id로 JOIN 조회
    # → agent별로 묶어 AgentOutput 형태로 복원
```

```
    #   {agent, task_id, issues[], summary, execution_ms}
    ...

# dev2/nodes/aggregate.py — DB 우선, 없으면 payload 폴백
async def _load_results(state):
    try:
        db_results = await fetch_scan_results_by_task(state["task_id"])
    except Exception:
        db_results = []
    if db_results:
        return db_results                    # ① DB 릴레이 (기본 경로)
    return state.get("all_results", [])      # ② payload 폴백 (하위호환)
```

언제 쓰나요?

- **적합한 상황**:
  - 에이전트들이 **이미 같은 DB를 공유**하고, 데이터를 어차피 저장해야 할 때
  - 받는 쪽이 보내는 쪽의 **실행이 끝난 뒤** 동작할 때 (비동기·시차 OK)
  - 중간 오케스트레이터가 큰 결과를 들고 다니는 부담을 줄이고 싶을 때
- **부적합한 상황**:
  - **실시간/저지연**이 중요할 때 (DB 왕복은 메모리 전달보다 느림)
  - 보내는 쪽 저장이 끝나기 **전에** 받는 쪽이 조회하면 **빈 결과** → 타이밍 의존성 발생
  - DB를 공유하지 않는 외부 팀/서비스 간 통신

코드 흐름 (단계별)

```
1. Dev2 store_node
     → scan_executions / scan_issues / scan_summaries 에 INSERT (task_id 붙여서)

2. Dev4 aggregate_node 시작
     → _load_results(state) 호출

3. _load_results
     → fetch_scan_results_by_task(task_id) 로 DB 조회
     → 결과 있으면 그걸 사용 (DB 릴레이) ✅
     → 없거나 조회 실패면 state["all_results"] (payload) 폴백

4. aggregate_node
     → 받아온 결과를 severity 순 정렬 → 다시 DB 저장 → M4로 push
```

프로젝트에서는

파일	역할
`dev2/db.py` `fetch_scan_results_by_task()`	task_id로 scan_issues를 조회해 AgentOutput 형태로 **복원**. JSONB 컬럼은 `_jload()`로 안전하게 파싱

파일	역할
`dev2/nodes/aggregate.py` `_load_results()`	**DB 우선 + payload 폴백** 분기. DB 릴레이의 진입점
`dev2/nodes/aggregate.py` `aggregate_node()`	복원한 결과를 취합·정렬·저장

검증(실제 동작 확인): 라이브 Cloud SQL에서 Dev2 실행 → DB 적재 후, `aggregate_node`를 **빈 payload(`all_results=[]`)** 로 호출했더니 task_id로 DB에서 6개 이슈를 가져와 정상 취합. `code_snippet`·`name_of_function` 같은 필드도 DB 왕복 후 보존됨. (`test_aggregate.py` 10 passed)

---

① code_snippet 버그 — Windows 경로 구분자

한 줄 요약

같은 파일인데 한쪽은 `C:\a\b.py`(역슬래시), 다른 쪽은 `C:/a/b.py`(슬래시)로 써서 컴퓨터가 "다른 파일"로 착각해 코드 조각을 못 찾던 버그입니다.

비유

같은 사람인데 한 명부엔 "홍길동", 다른 명부엔 "홍 길동"(띄어쓰기)이라 적혀 있으면 컴퓨터는 다른 사람으로 봅니다. 띄어쓰기를 통일해주면 같은 사람으로 인식합니다.

왜 / 어떻게

- 도구마다 파일 경로 표기가 달랐습니다: **pylint = 역슬래시**, **bandit = 슬래시**.
- `pr_diff` 헤더(`+++ b/...`)와 **문자열 그대로 비교**하니 구분자만 달라도 매칭 실패 → `code_snippet`이 항상 `"snippet_unavailable"`.
- **수정**: 비교 직전에 양쪽 경로를 `.replace("\\", "/")`로 **정규화**.

```
# dev2/nodes/concept_mapper.py — extract_snippet() / _collect_file_lines()
target_file = target_file.replace("\\", "/")   # 역슬래시 → 슬래시 통일
...
file_path = raw_line[6:].replace("\\", "/")
```

교훈: 경로 비교는 항상 구분자를 통일하고 하자. (Windows ↔ Unix 혼용 환경의 단골 버그)

---

② name_of_function — 함수명 자동 추출 (AST)

한 줄 요약

"이 버그가 **어느 함수 안에서** 났는지"를 코드 구조를 분석해 자동으로 알아내는 기능입니다.

비유

책에서 한 문장이 **몇 장(章)에 속하는지** 찾는 것과 같습니다. 목차(구조)를 보고 그 문장을 감싸는 가장 가까운 챕터 제목을 골라냅니다.

왜 / 어떻게

- M4(학습 피드백)가 "어느 함수에서 실수했는지" 알면 더 정확한 피드백을 줄 수 있습니다.
- **Python**: `ast` 모듈로 코드를 **구조(트리)** 로 파싱해, 문제 라인을 포함하는 **가장 안쪽 함수**(`FunctionDef`)를 찾습니다. 중첩 함수면 제일 안쪽을 고릅니다.
- **비-Python / 파싱 실패**: 정규식으로 문제 라인 **위쪽을 거슬러 올라가며** `def`/`function`을 탐색.
- 함수 밖(모듈/클래스 본문)이면 빈 문자열 `""`.

```
# dev2/nodes/concept_mapper.py
def extract_function_name(pr_diff, target_file, target_line) -> str:
    # Python → AST로 정확히, 실패 시 정규식 fallback, 함수 밖이면 ""
```

핵심 개념: **AST(Abstract Syntax Tree, 추상 구문 트리)** = 코드를 글자가 아니라 **구조(트리)** 로 본 것. 정규식이 "글자 패턴"이라면 AST는 "문법 구조"라 훨씬 정확합니다.

복습 퀴즈 (3문항)

1. **(이해)** payload 방식과 DB 릴레이 방식의 가장 큰 차이는 무엇인가요? 그리고 DB 릴레이에서 받는 쪽(Dev4)이 보내는 쪽(Dev2)의 결과를 찾을 때 쓰는 **공통 키**는 무엇인가요?

2. **(적용)** DB 릴레이는 "받는 쪽이 보내는 쪽 저장이 끝나기 전에 조회하면 빈 결과가 나온다"는 약점이 있습니다. 우리 코드(`_load_results`)는 이 상황(또는 DB 연결 실패)에서 어떻게 동작하나요? (힌트: 폴백)

3. **(비교)** `code_snippet` 버그는 "글자 그대로 비교"라서 생겼고, `name_of_function`은 "글자 패턴(정규식)" 대신 "문법 구조(AST)"를 우선 씁니다. 왜 함수명 찾기에는 정규식보다 AST가 더 정확할까요?

연관 개념

- **AgentOutput 계약** → 에이전트끼리 주고받는 표준 JSON 형식 (CLAUDE.md §5-A). DB 릴레이는 이 형식을 DB에서 복원함.
- **JSONB 복원(`_jload`)** → DB의 JSON 컬럼을 dict/list로 되돌리기. 드라이버가 문자열로 주든 이미 파싱해 주든 안전 처리.
- **하위호환 폴백** → 새 방식(DB 릴레이)을 넣되 기존 방식(payload)도 살려둬서 갑자기 안 깨지게 하는 설계.
- 더 깊게: `docs/learning/ai/dev2-pipeline2.md`, `docs/learning/ai/store_node_patterns.md`